Trusted Execution Environment (TEE)

Mengjia Yan

Spring 2026

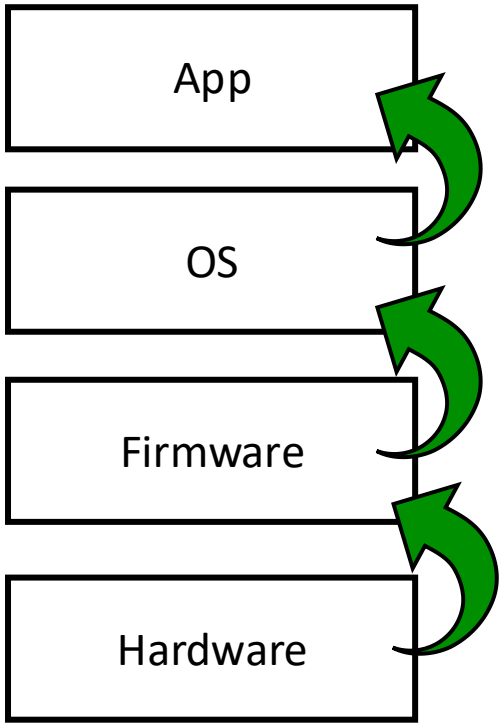
Recall: The Trust Bootstrapping Problem

To verify the App

→ need to trust the OS that verifies it

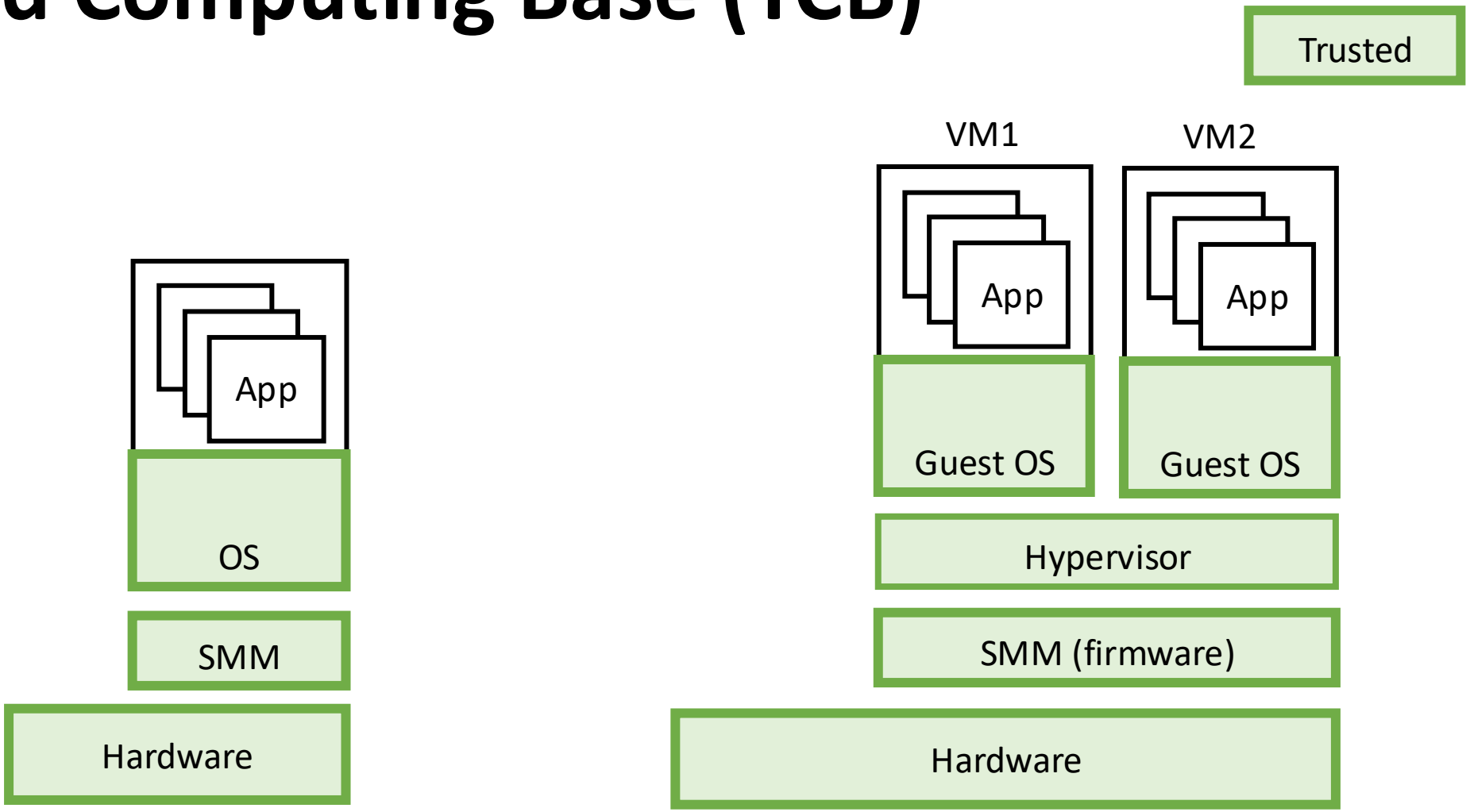
→ need to trust the firmware that verifies OS

→ need to trust the hardware that verifies the firmware



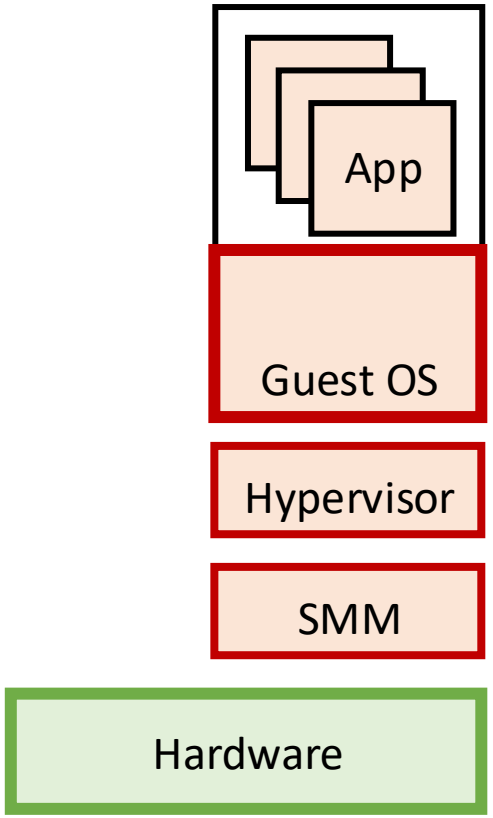
- **Verification requires a trusted starting point: Root of Trust (RoT)**

Trusted Computing Base (TCB)



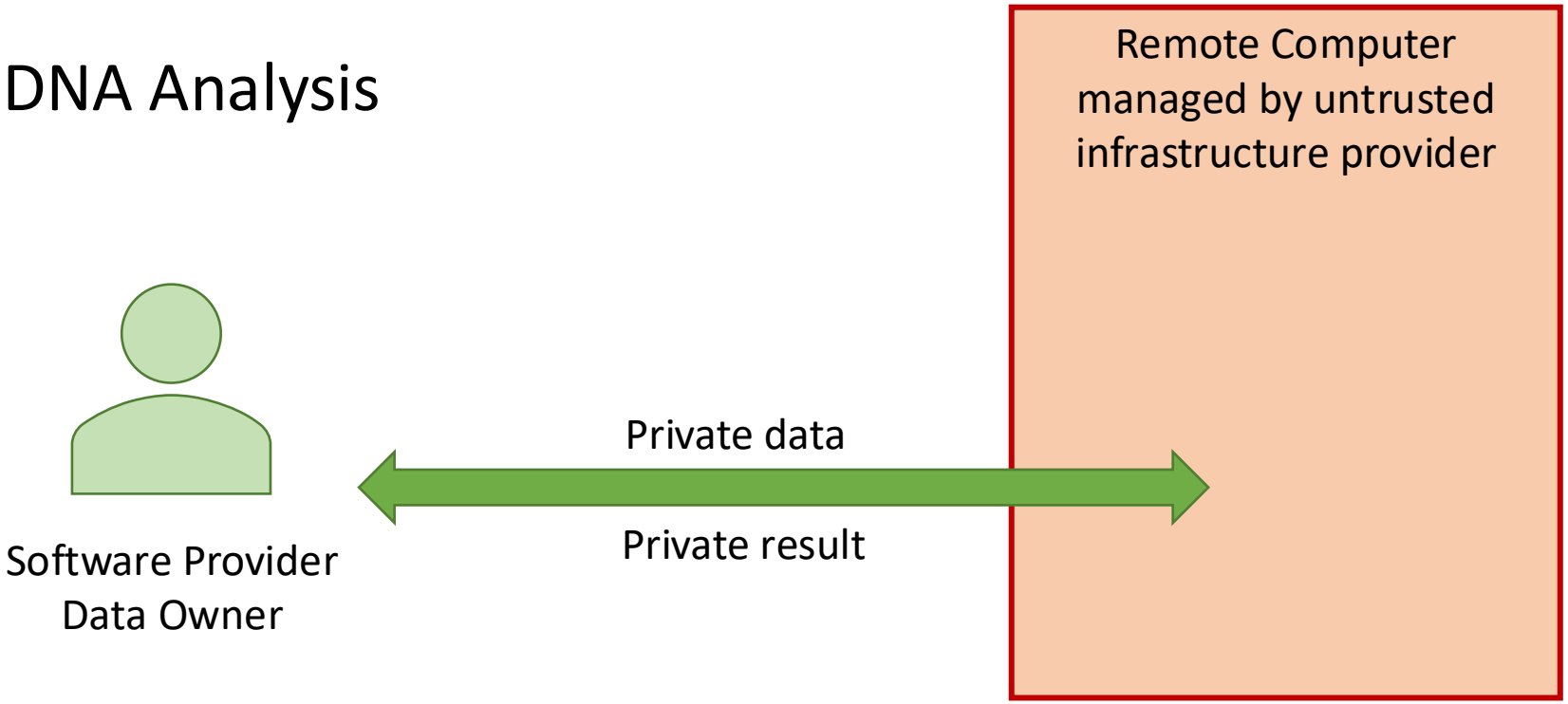
Shrink TCB. Why?

- Software bugs
  - SMM-based rootkits
  - Xen 150K LOC, 40+ vulnerabilities per year
  - Monolithic kernel, e.g., Linux, 17M LOC, 100+ vulnerabilities per year
- Remote Computing
  - Remote computer and software stack owned by an untrusted party



Secure Remote Computing

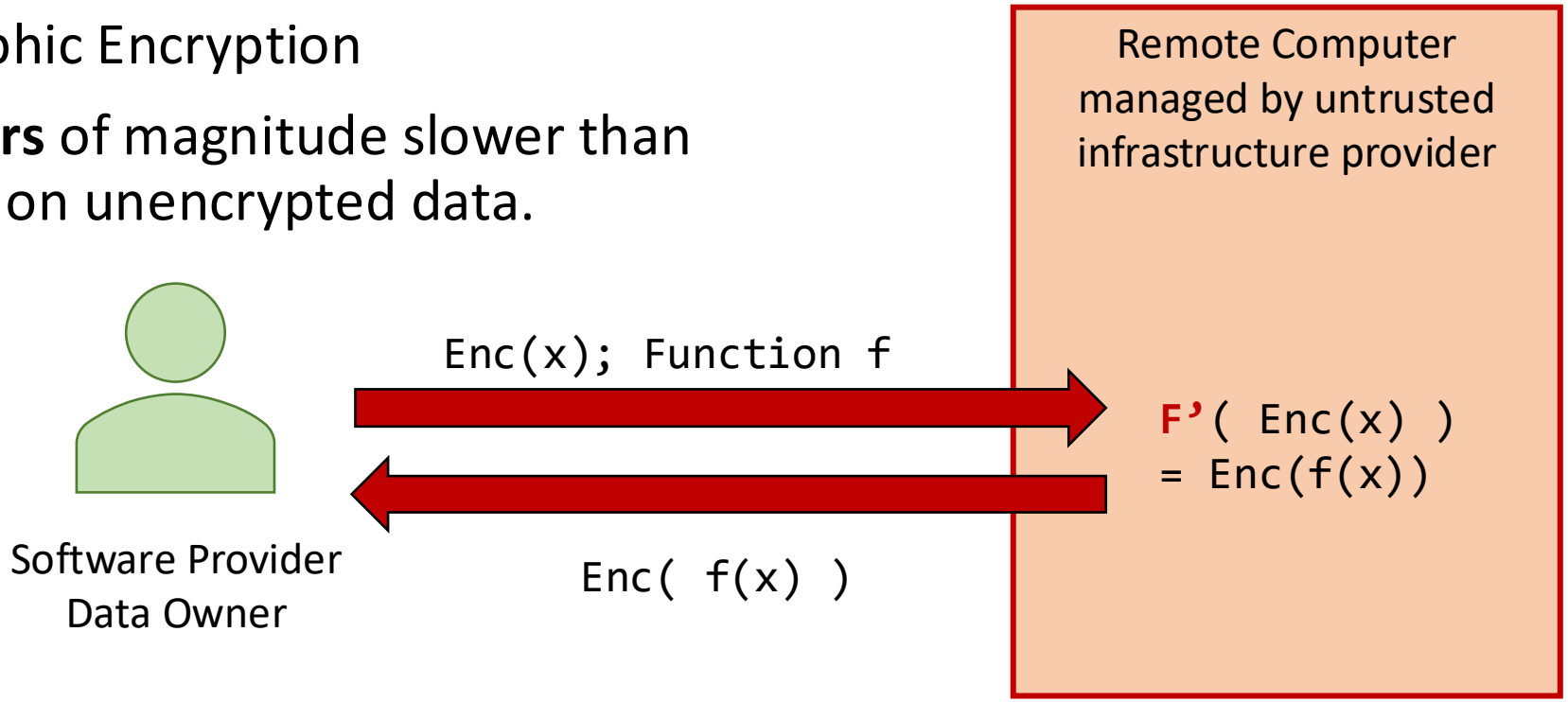
- Example: DNA Analysis



How to keep my data private without trusting the host OS/hypervisor/SMM?

Potential Solutions

- Homomorphic Encryption
- **4 to 5 orders** of magnitude slower than computing on unencrypted data.

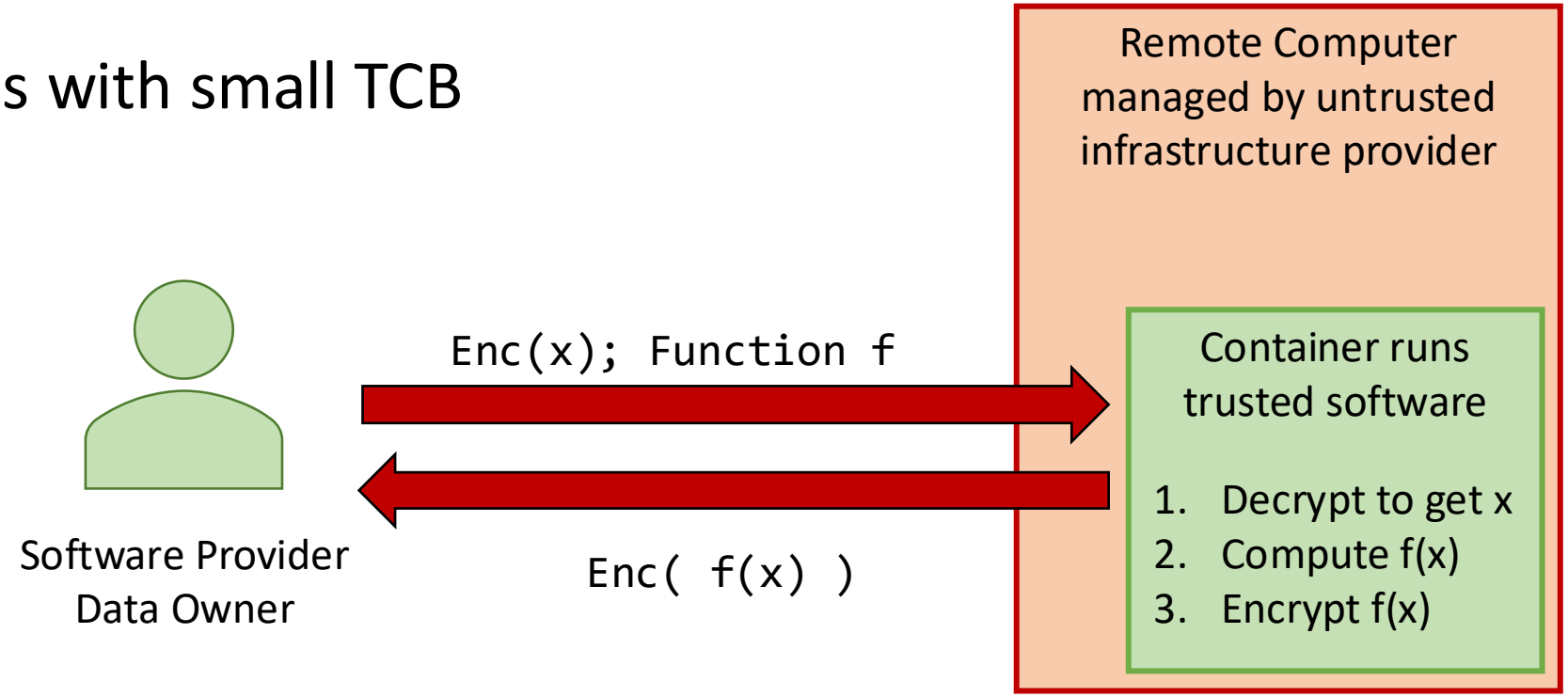


- Performance? Accelerators?

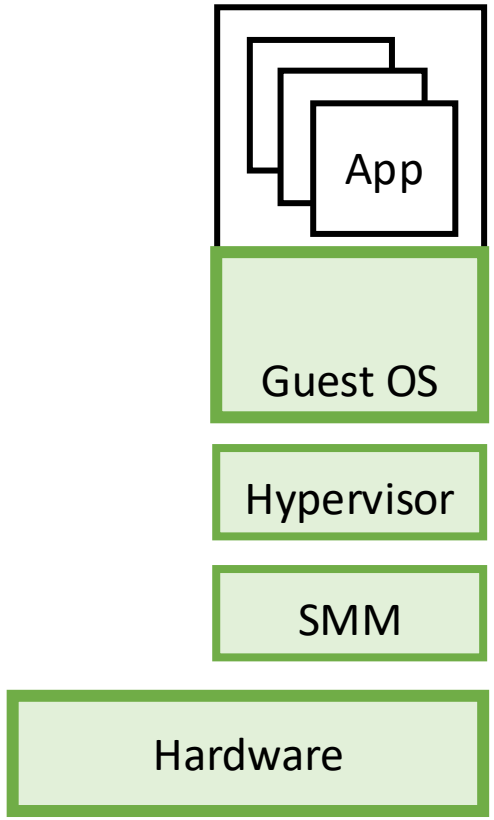
e.g., F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption; Axel Feldmann, Nikola Samardzic et al. MICRO'21

Potential Solutions

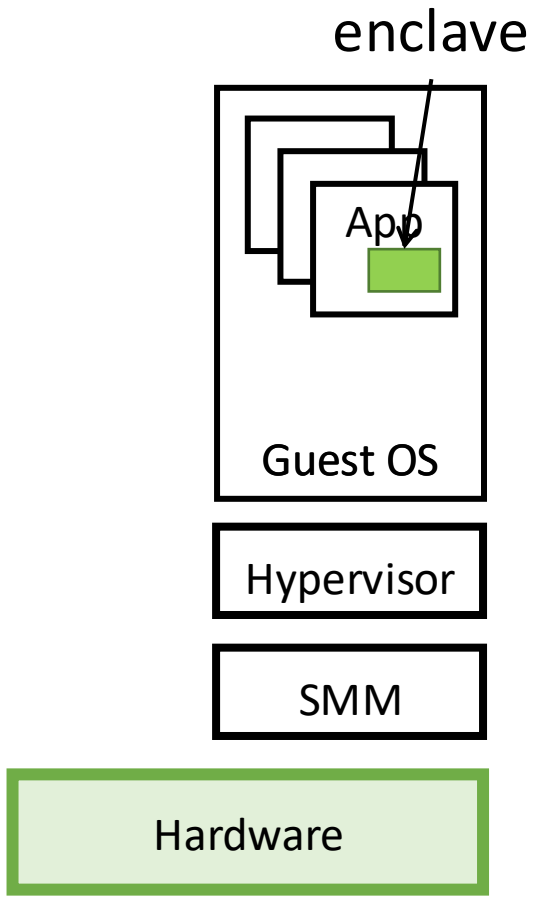
- Build TEEs with small TCB



Protection Granularity & TCB Size



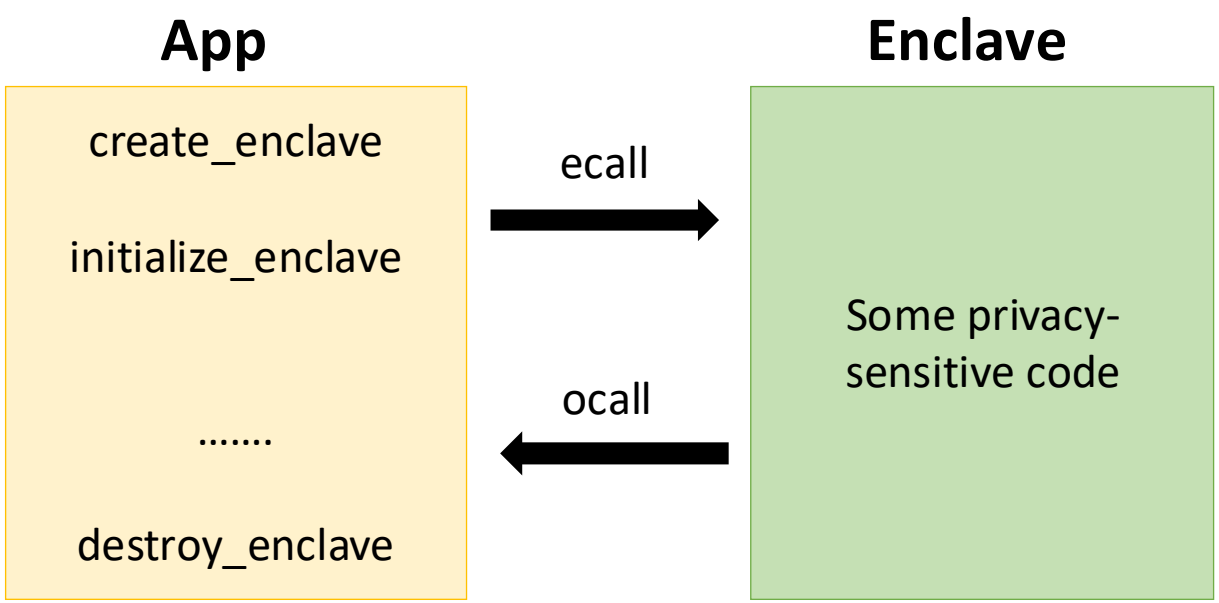
Traditional System



Intel SGX

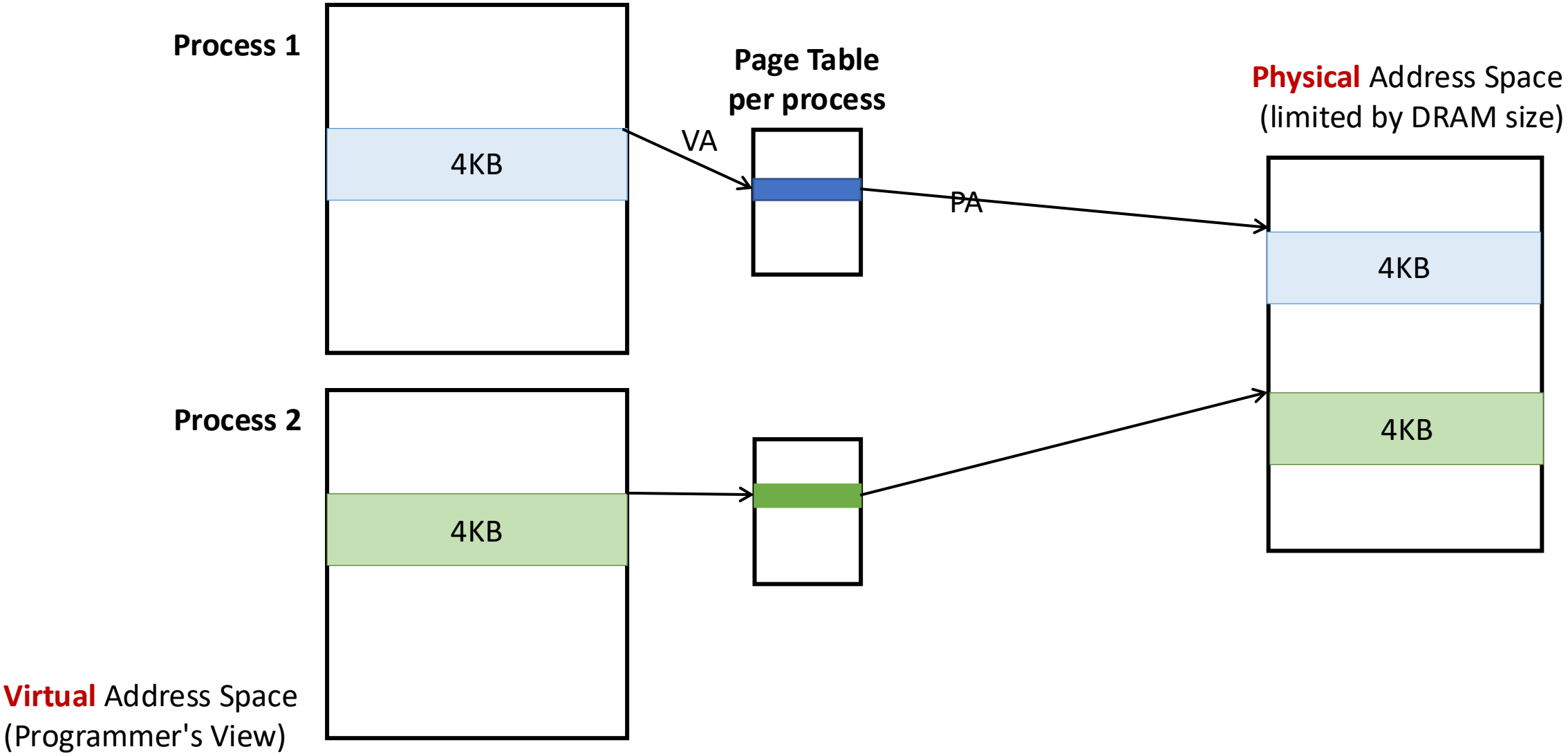
SGX Enclave Programming Model

- Examples from: https://github.com/intel/linux-sgx

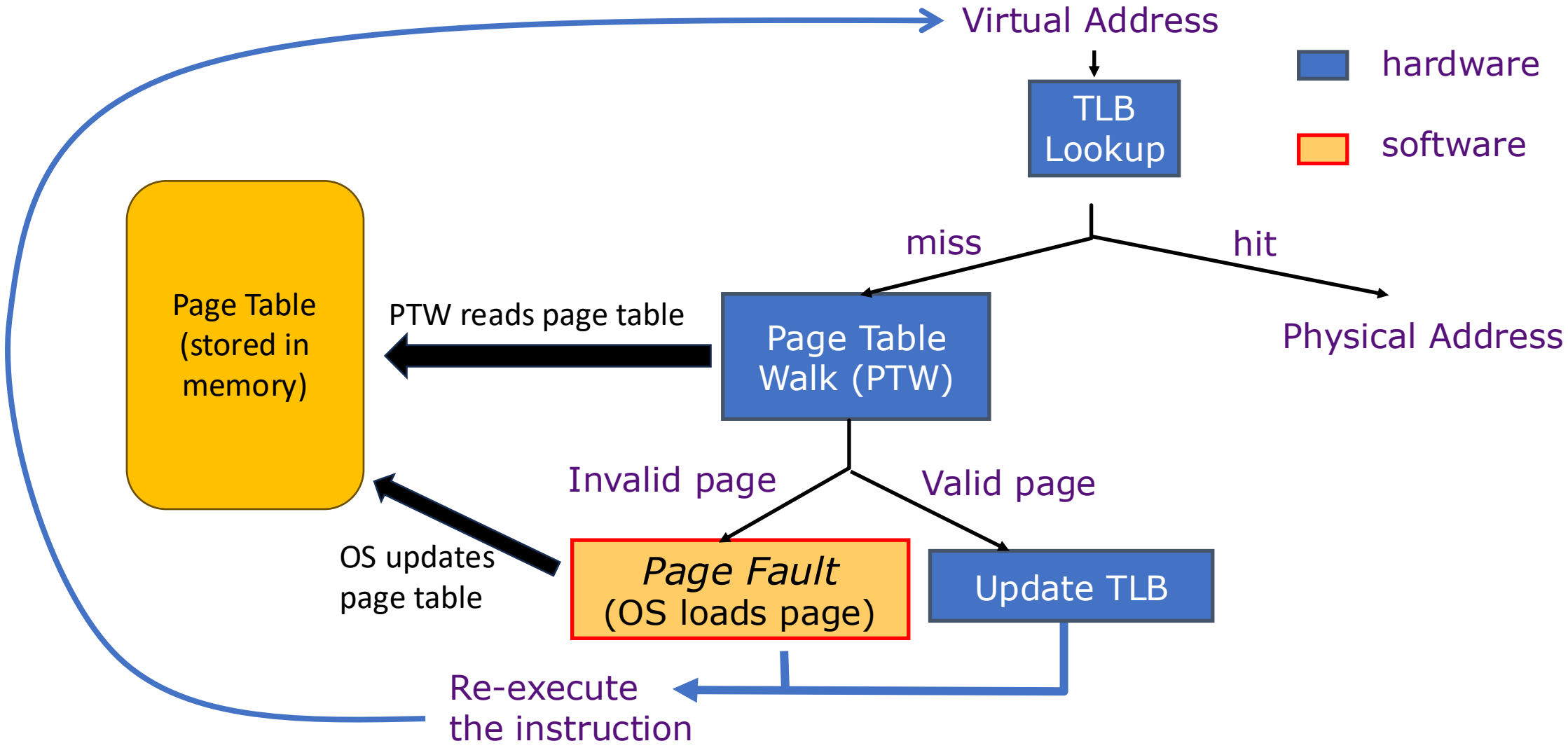


Case Study: Memory Management in Intel SGX

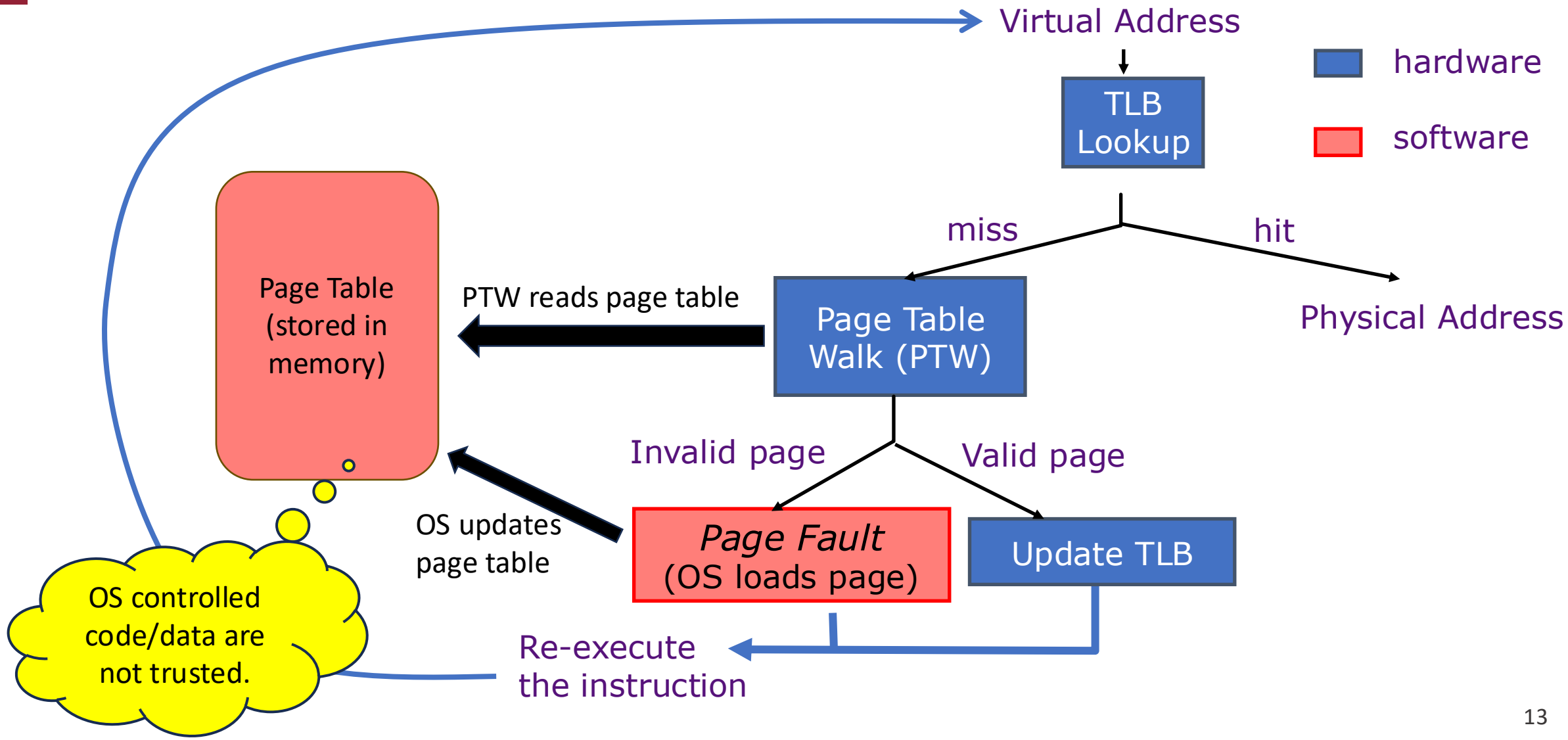
Recap: Virtual Address & Address Mapping



Recap: Address Translation



Under the TEE Threat Model



Intel SGX Overview

Security goal:

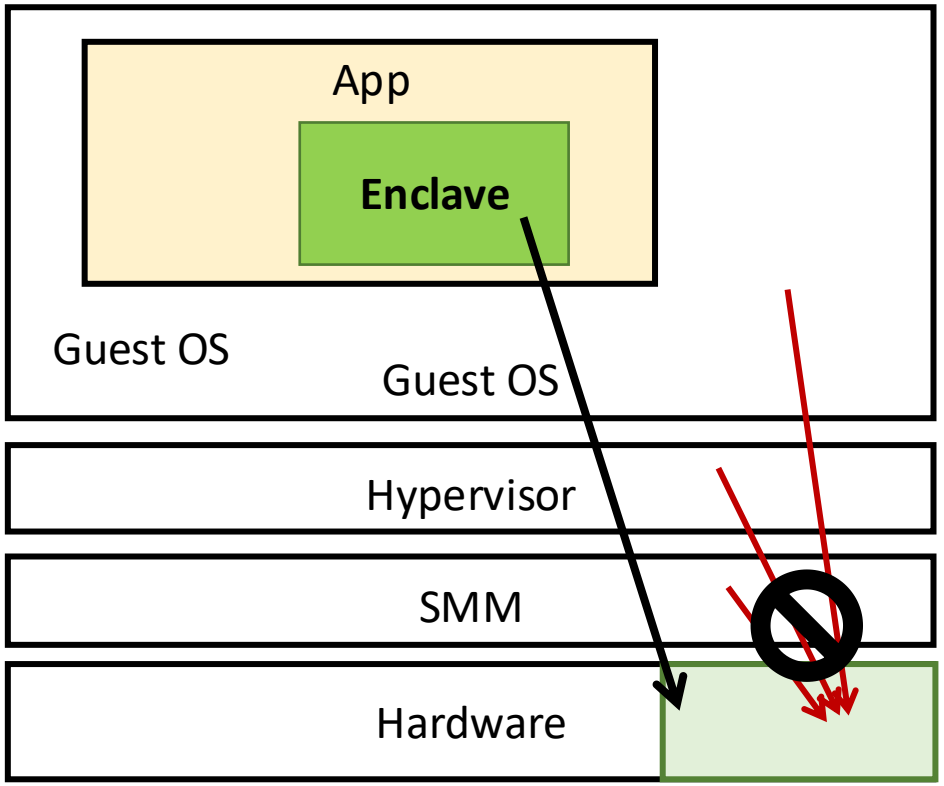
- **Isolation** between the enclave and the rest of the system **without trusting** OS-managed page tables

Concretely:

- Enclave code/data map to the PRM
- Different enclaves access their own memory regions
- Other code cannot access the PRM

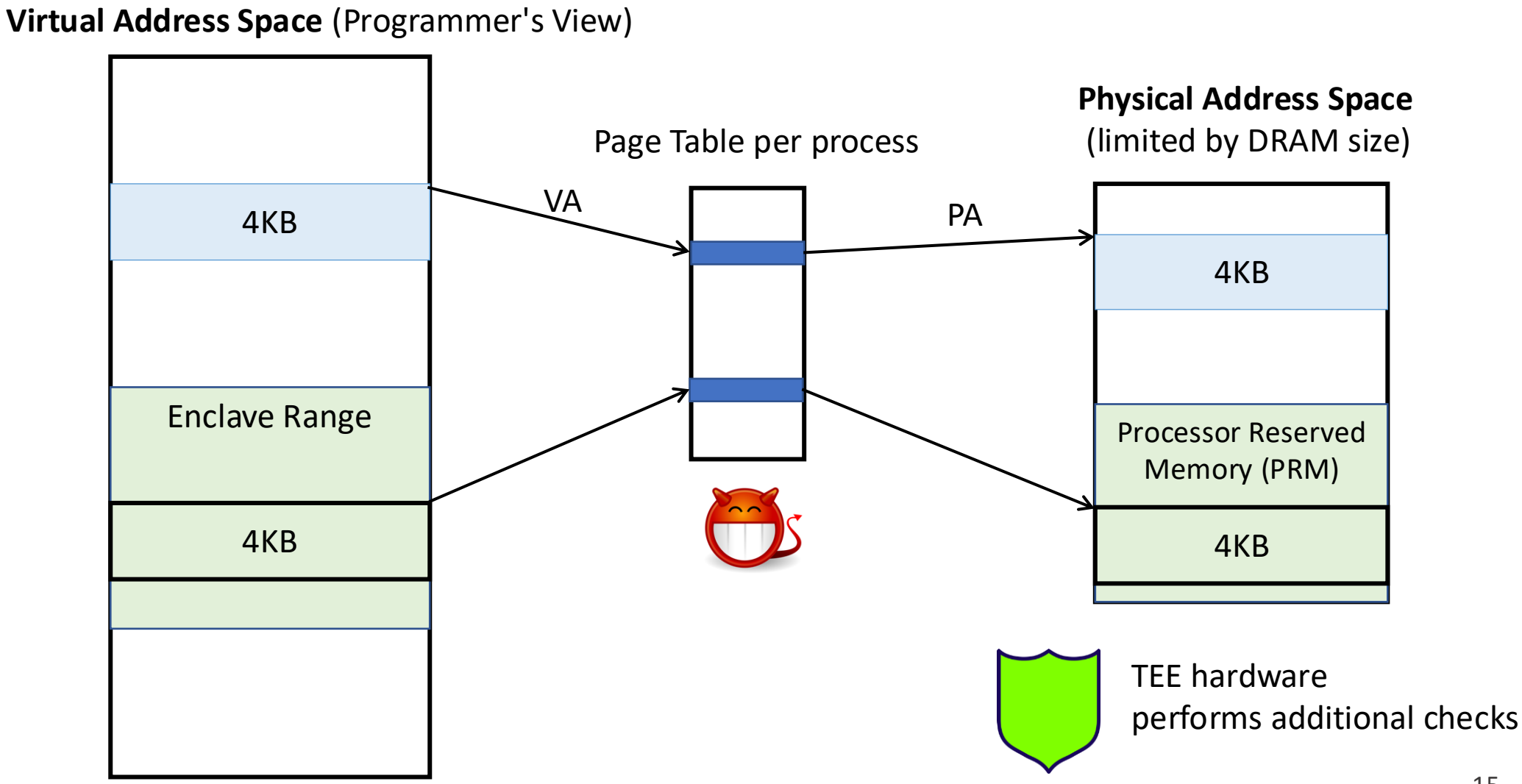
Approach:

- Hardware (e.g., PTW) enforces security invariants



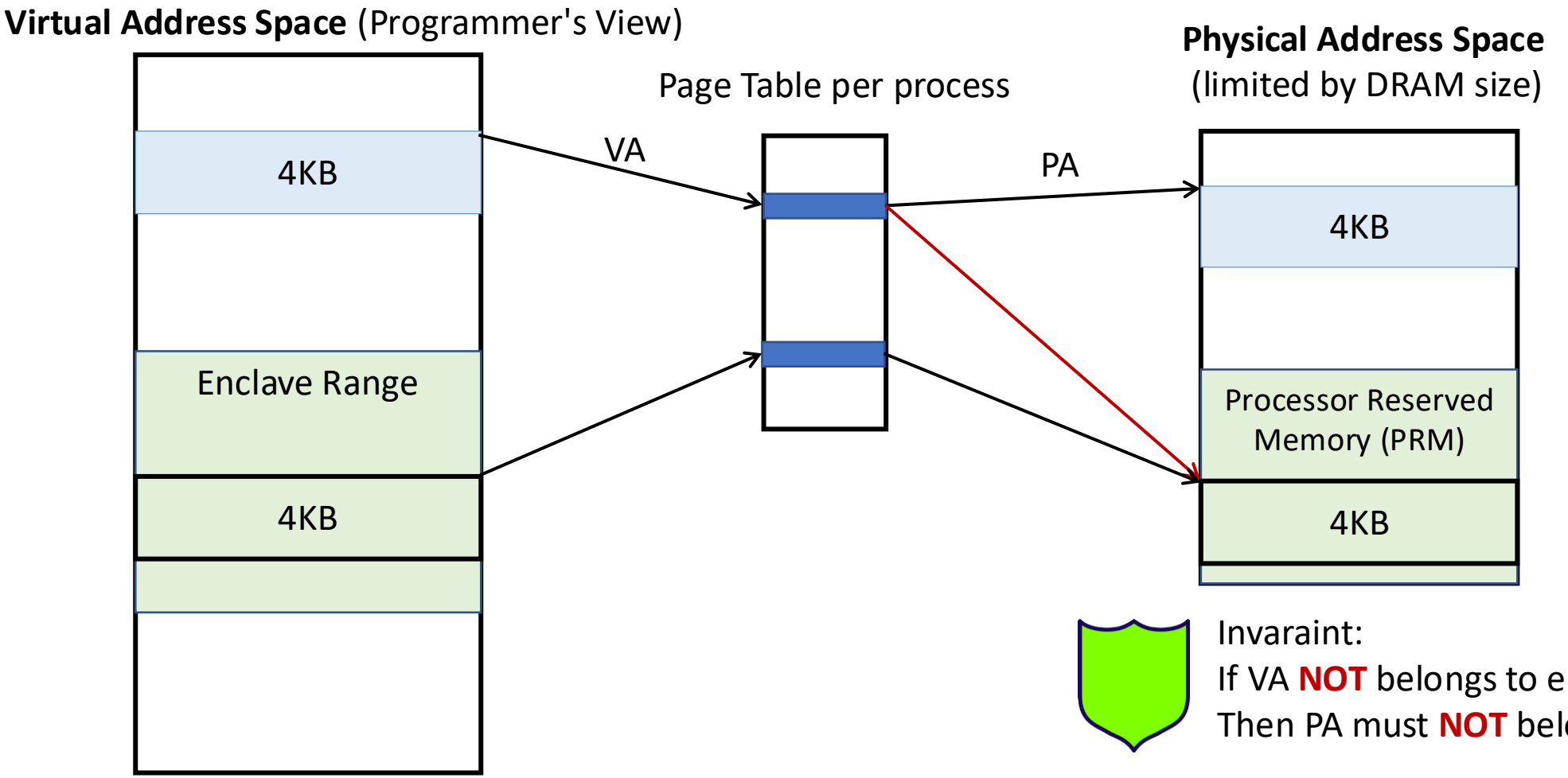
Processor Reserved Memory *(PRM)*

Intel SGX Address Translation Overview



Attack #1:

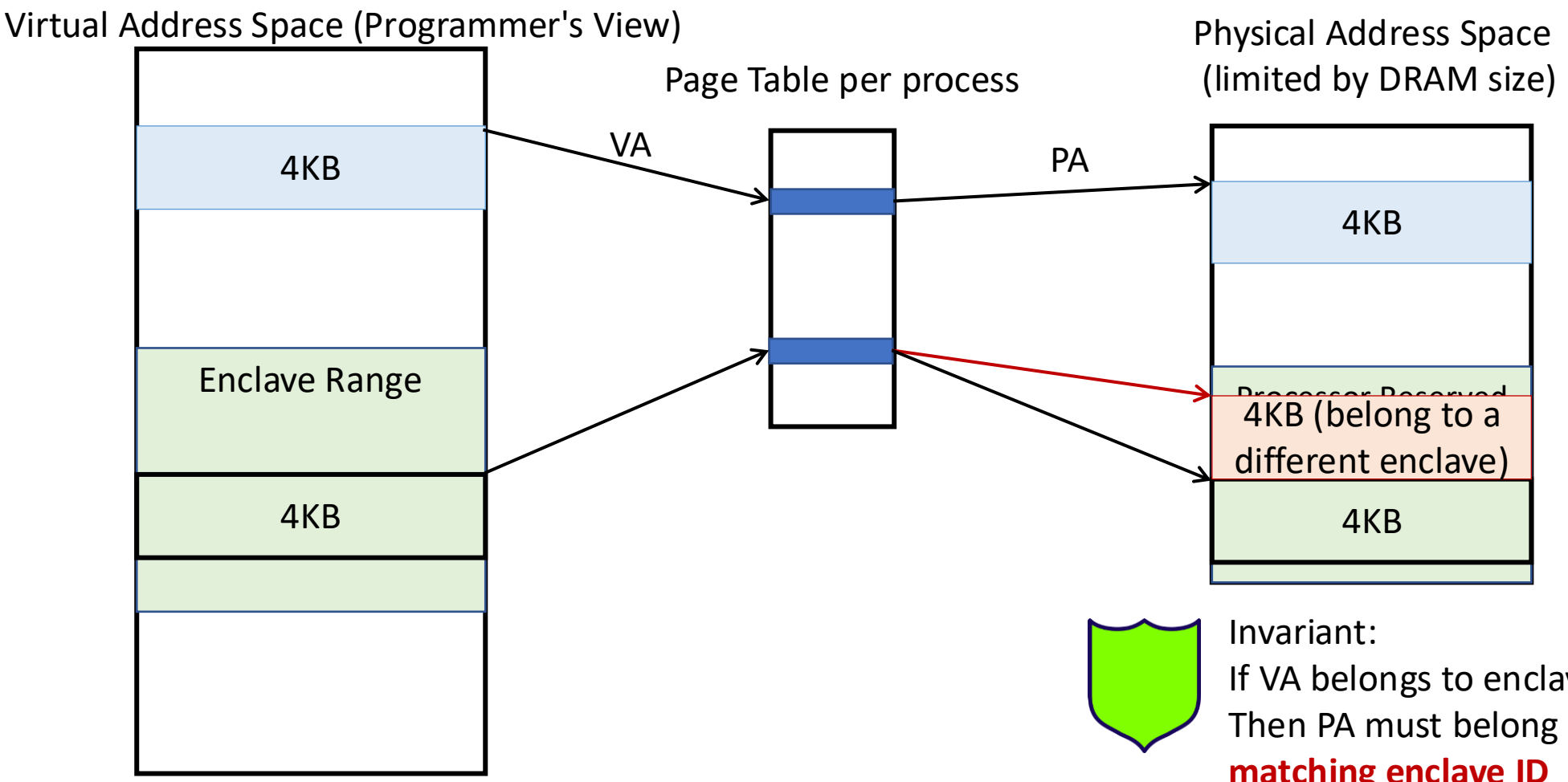
Map an OS page to the RPM; Map an enclave outside of the RPM



Invaraint:
If VA **NOT** belongs to enclave range,
Then PA must **NOT** belong to PRM

Attack #2:

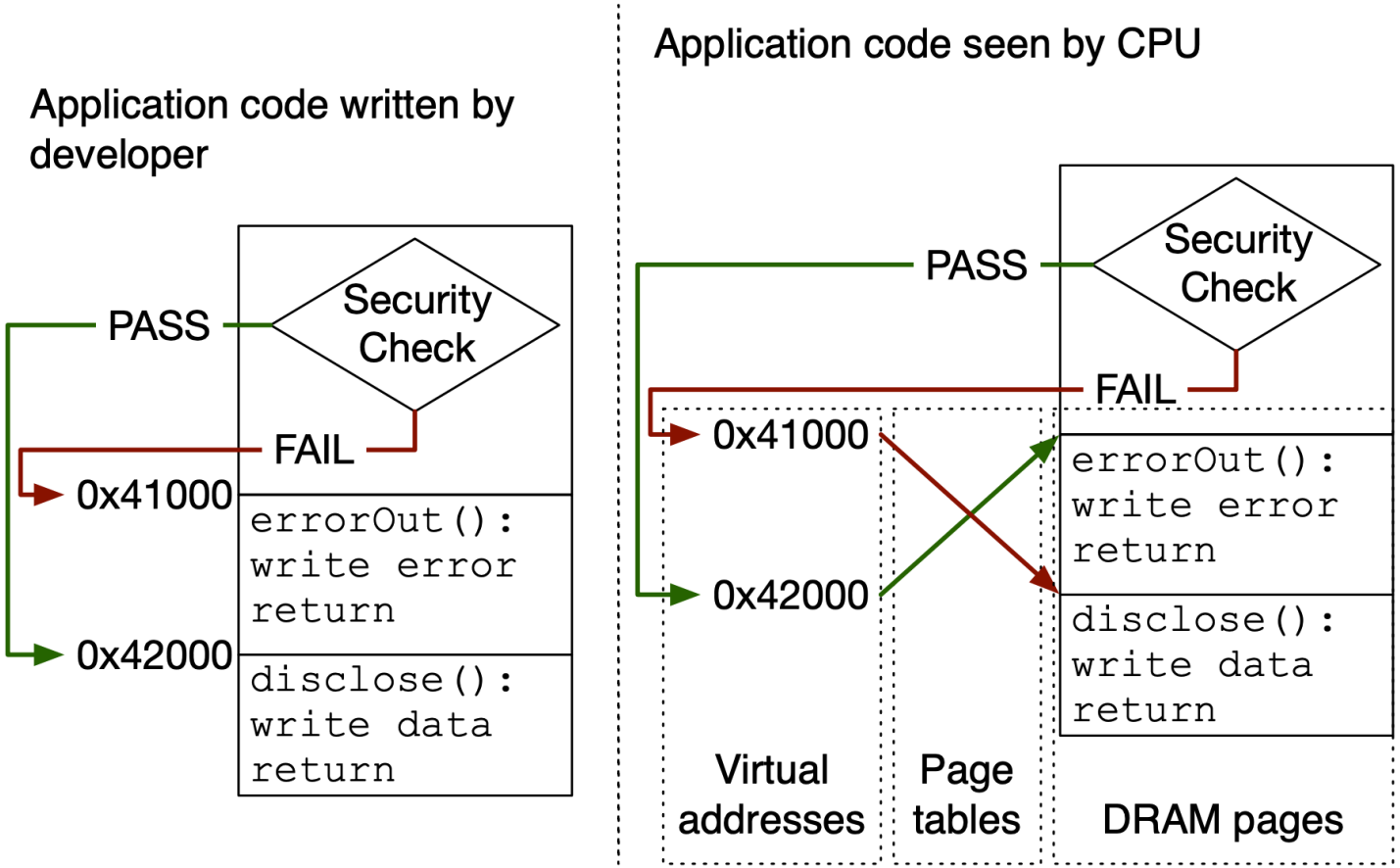
Map an attacker enclave's page to a protected enclave's page



Invariant:
If VA belongs to enclave range,
Then PA must belong to PRM
matching enclave ID

Attack #3:

Mess up the mapping of pages belonging to the same protected enclave



Solution: Inverted Page Table

For each page in the PRM, remember the inverted mapping:

<PPN> → <VPN, Enclave ID>

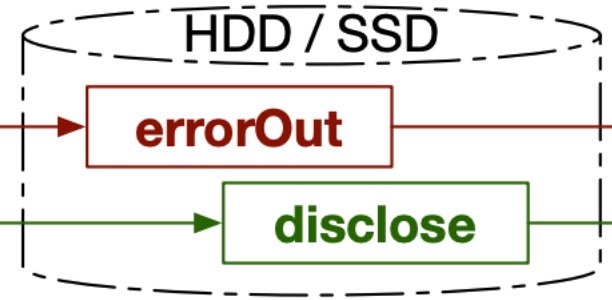
Keep the reversed page table in PRM, so privilege software cannot modify

Attack #4:

Modify the contents of a page while it is swapped out of the PRM

Page tables and DRAM before swapping

Virtual	Physical	Contents
0x41000	0x19000	errorOut
0x42000	0x1A000	disclose

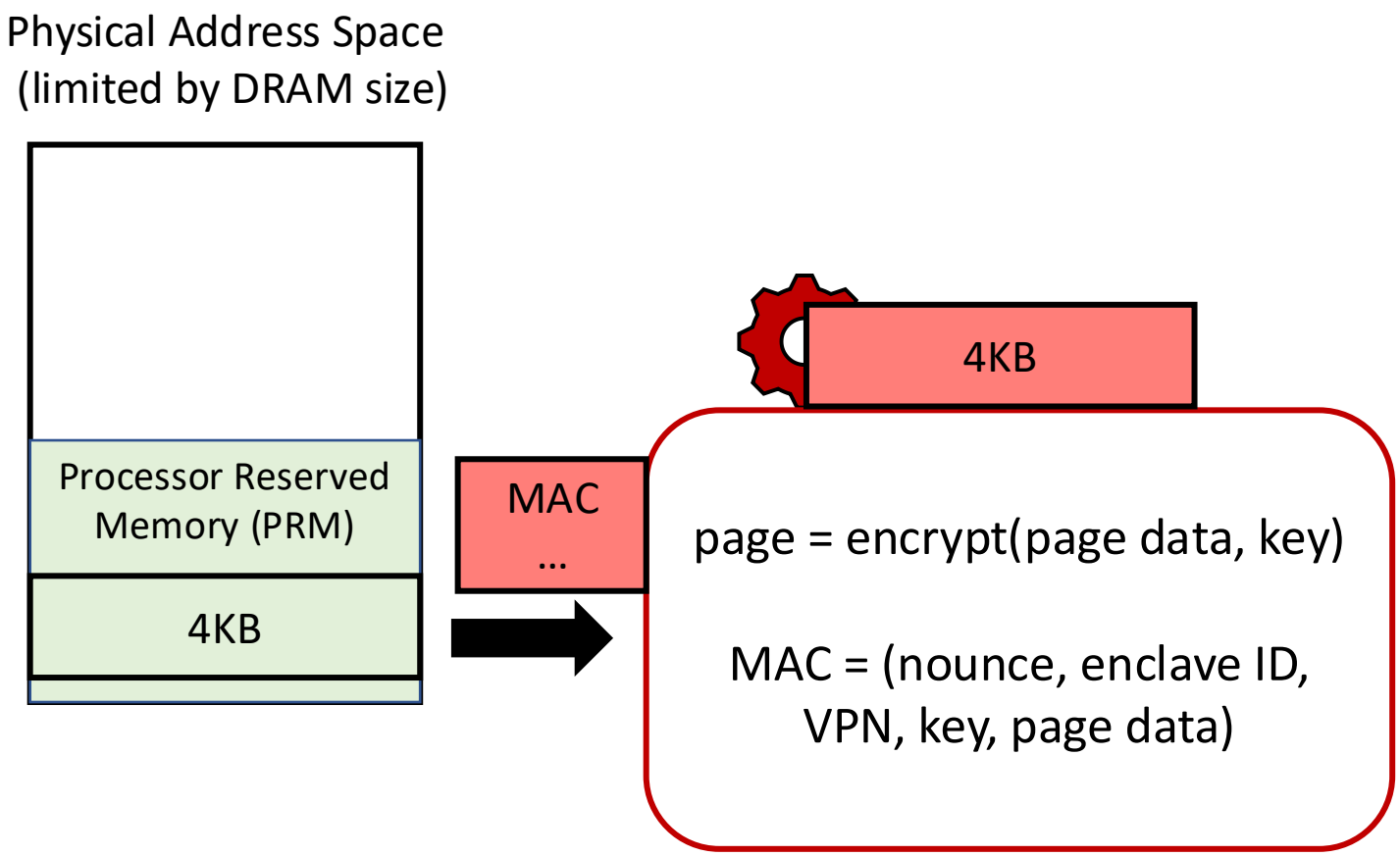


Need to bind the virtual address mapping with the page content.

Page tables and DRAM after swapping

Virtual	Physical	Contents
0x41000	0x19000	disclose
0x42000	0x1A000	errorOut

Solution: Page Encryption and Authentication



Summary: SGX Memory Management

- #1: Maintain a inverted page table and check after every address translation

  Physical page in PRM -> (enclave ID, virtual page number)

- #2: Encrypt/decrypt upon page swap to non-PRM region

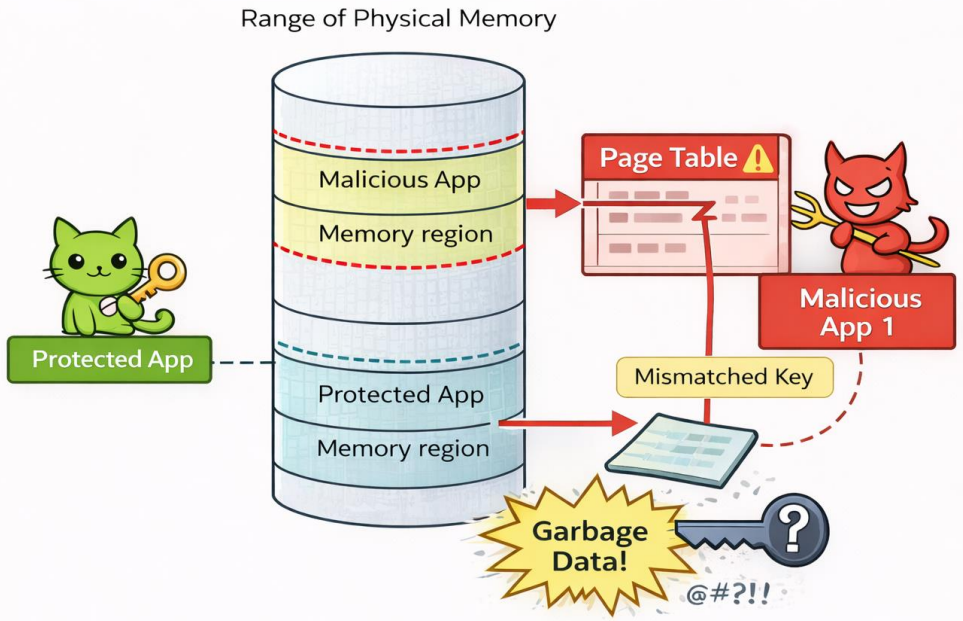
  (nounce, enclave ID, virtual page number, key, page content) → MAC

- #3: Keep TLB state up-to-date

  (We did not discuss this item in detail. Feel free to refer to the reference materials)

Alternative Solutions

- Naïve idea:
  - Let the trusted component handle page management
  - Problem: Large TCB
- Keystone's approach:
  - Leverage RISC-V's PMP registers to enforce coarse-grained isolation
  - Let the application to manage their page mappings
- AMD SEV:
  - Rely on encryption. But symmetric key vulnerability
  - Recent version introduces reverse page table



Illustrate AMD SEV working mechanism.

Generated by ChatGPT

Summary

- Understand the threat model
- Why it is challenging to achieve security in such a threat model
- Design tradeoffs between TCB size, flexibility, perf overhead, cost, etc.
  - Intel SGX, AMD SEV, ARM CCA, Keystone, Sanctum, Penglai, etc
- Recommended reading:
  - Intel SGX Explained; by Costan et al